

Comparative Analysis of Deep Learning Models used for Face Anti-Spoofing in Secure Biometric Authentication

1st Eshanya Padial
Department of Computer Science
Manipal University Jaipur
Jaipur, Rajasthan, India
eshanyapadial@gmail.com

2nd Jai Sachdeva
Department of Computer Science
Manipal University Jaipur
Jaipur, Rajasthan, India
sachdevajai319@gmail.com

Abstract— Face-based biometric authentication systems are increasingly used for identity verification but remain vulnerable to spoofing attacks such as printed images, digital displays, and deepfake content. This paper presents a foundational study for the development of a robust face anti-spoofing authentication system through a comparative analysis of deep learning architectures, including MobileNetV2, EfficientNetB0, and ResNet50. The study evaluates model performance on real and manipulated facial datasets to identify the most suitable architecture for real-world deployment. Experimental results demonstrate that MobileNetV2 achieves the best balance between accuracy and computational efficiency, making it a strong candidate for practical implementation. The findings of this research establish a baseline framework and provide critical insights for designing secure, real-time anti-spoofing authentication systems, forming the basis for future system-level development and integration.

Keywords— Face Anti-Spoofing, Deep Learning, MobileNetV2, ResNet50, EfficientNetB0, Biometric Authentication, Deepfake Detection, Liveness Detection, Transfer Learning, Convolutional Neural Networks

I. INTRODUCTION

Biometric authentication has emerged as a dominant paradigm for secure identity verification in modern digital systems [23], [40]. Among various biometric modalities, facial recognition systems have gained widespread adoption due to their non-intrusive nature, ease of deployment, and compatibility with consumer-grade hardware such as

surveillance systems and smartphones [21]. Yet, regardless of their benefits, such systems are predisposed to spoofing attacks, when attackers try to evade the authentication by printed pictures, replayed videos, or artificially generated deepfakes [25], [42].

The threat is growing with the growing complexity of generative models, in particular Generative Adversarial Networks (GANs) [5]. Deepfake technologies can generate very realistic face images which are often difficult to distinguish between the real images, thus posing severe security threats [43], [44]. As a result, the necessity to have strong face anti-spoofing mechanisms has become pivotal [8], [11].

The problem discussed in this research is to differentiate between real and spoofed or deepfake representations by using a supervised deep learning model. The system is developed to be a binary classification task, whereby the input facial images will be classified as either real or fake. Besides the classification, the system also includes a verification system that is based on Euclidean distance between facial embeddings and registered identities, which is an extra security in authentication.

This work exploits transfer learning through the state-of-the-art convolutional neural networks, unlike the traditional methods of incorporating the handcrafted features of texture descriptors or motion cues in designing the concept of touch [22], [24]. Three architectures MobileNetV2 [2], EfficientNetB0 [4], and ResNet50 [3] are chosen to be comparatively evaluated in terms of their diversity in architecture, performance, and their applicability to the real-life deployment environment.

The main aim of the research is to examine how effectively these architectures detect spoofing attacks yet at the same time be computationally efficient. The results will be used

to help in the creation of secure, scalable, and real-time face authentication systems that can withstand the modern adversarial attacks.

II. LITERATURE REVIEW

The face anti-spoofing field has developed over time and shifted towards more sophisticated methods of deep learning as compared to the earlier image processing methods. Initial techniques were mainly based on handcrafted characteristics like Local Binary Patterns (LBP), Histogram of Oriented Gradients (HOG) and other texture-based descriptors in order to detect discrepancies between genuine and spoofed facial images [24], [22]. Though these methods offered some basic understanding, they were not very robust to different illumination conditions and pose variations as well as environment noise [25].

Introduction of deep learning was a revolutionary move in this field. The introduction of convolutional neural networks (CNNs) made it possible to extract features automatically so that models can learn complex and discriminative features directly by using raw images [6], [7]. Deep hierarchical feature learning architectures, like VGG, showed that such architectures can greatly enhance the results in face recognition and anti-spoofing problems [6]. These methods removed the reliance on handcrafted characteristics and offered increased flexibility over datasets.

More developments were made with more advanced and profound architectures. ResNet50 [3], a deep neural network, solved this issue by taking advantage of the residual learning concept, which overcomes the degradation issue of the deep neural networks by adding skip connections that enable identity mappings. This innovation allowed training much deeper networks without being afflicted with a vanishing gradient problem. Nonetheless, the added level of depth and number of parameters led to increased complexity of computation, and thus these models were less applicable in real time or resource poor applications.

Lightweight architectures like MobileNetV2 [2] were developed in order to tackle the challenges of efficiency. MobileNetV2 employs inverted residual blocks, depthwise separable convolutions to significantly lower computational expenses and model size without compromising on accuracy [1]. This renders it especially appropriate to be used in real time face authentication systems on mobile and embedded systems.

EfficientNet [4] further improved the model design with a compound scaling strategy which balances network depth, width and resolution systematically. The baseline model, EfficientNetB0, is one that offers an optimal balance of accuracy and efficiency. Its performance though is very much dependent on adequate training data and long time of training which can restrict its application in limited experimental conditions.

Recent studies have investigated further-developed strategies to enhance the anti-spoofing efficiency. In temporal modelling, dynamic facial expressions like eye blinking, head motion, and micro-expressions are analyzed, which are hard to imitate in spoofing attacks [35]. Moreover, architectures based on transformers like Vision Transformers (ViT) have also shown great potential to extract global contextual information [12]. Multimodal methods that combine RGB images and depth, infrared or thermal have also demonstrated high resilience to advanced attacks of spoofing [38].

Although these developments have been made, there are still a number of challenges that are not overcome. These are the ability to achieve generalization on varied data sets, able to identify previously unknown or new forms of attacks and being able to perform in real-time in a practical deployment setup [11], [43]. Overcoming these issues is important in coming up with dependable and scalable biometric authentication systems. The present research paper is relevant to the available literature because it offers systematized comparative research on three popular CNN models MobileNetV2, EfficientNetB0, and ResNet50 - within the framework of a single experiment. The goal is to gauge their performance in terms of identifying spoofing attacks and strike a balance between computational efficiency to form a solid basis in face anti-spoofing authentication systems in the real world.

TABLE I. Evolution of Face Anti-Spoofing Techniques

Approach Type	Techniques	Advantages	Limitations
Traditional Methods	LBP, HOG, Texture Analysis [24], [22]	Low computational cost, simple implementation	Poor robustness, limited generalization
Deep Learning (CNN)	VGG, CNN-based models [6], [7]	Automatic feature extraction, higher accuracy	Requires large datasets
Advanced Methods	Temporal, Transformer, Multimodal [35], [12], [38]	Improved robustness and detection capability	High complexity and resource demand

TABLE II. Comparative Overview of CNN Architectures

Model	Key Feature	Parameters	Advantages	Limitations
MobileNetV2 [2]	Depthwise separable convolutions	~3.4M	Lightweight, fast, suitable for real-time use	Slightly lower representation capacity
ResNet50 [3]	Residual connections	~25M	Strong feature extraction capability	High computational cost
EfficientNetB0 [4]	Compound scaling	~5.3M	Balanced accuracy and efficiency	Requires longer training

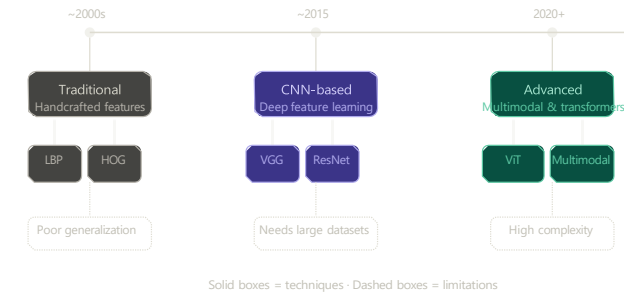


Fig.1. Recent Advancements in Anti-Spoofing

III. METHODOLOGY AND IMPLEMENTATION

The proposed system is modelled as a deep learning-based face anti-spoofing system that can help to identify authentic face inputs and spoofed or deep fake images. The methodology merges data-preparation, preprocessing, transfer-learning, model-training, evaluation and system-level-deployment into a single pipeline, which is reproducible and scalable to real-world applications.

A. Overall System workflow.

The process is systematic with a series of activities starting with data collection to ultimate analysis. Raw facial images are collected and organized and then undergo preprocessing and augmentation to provide uniform model input. Pretrained CNN architectures are then used to extract deep features, and binary classification is used to decide whether an input has been real or fake. The ultimate performance is appraised on a holistic set of evaluation measures.

TABLE I. System Workflow Stages

Stage	Description
Input Acquisition	Collection of facial images from dataset
Preprocessing	Image resizing and normalization
Feature Extraction	Deep feature extraction using CNN models
Classification	Binary classification (Real vs Fake)
Evaluation	Performance assessment using multiple metrics

B. Dataset Handling and Preparation

The data set comprises of facial images in two groups: real and fake. Images are loaded based on organised directories and with binary label (Fake = 0, Real = 1). Deterministic shuffling with a fixed random seed is

used to partition the data into training, validation and test subsets to achieve reproducibility and avoid bias in the model.

C. Data Preprocessing and Augmentation.

Input images are converted to tensors, all images are resized to 224 224 and their pixel values are brought to the range [0, 1] by scaling. Data augmentation methods, such as random horizontal flipping and brightness and contrast changes of $\pm 15\%$ and between the ranges of 0.85-1.15 respectively, are only used on the training set to replicate variability in the real world and enhance model generalization. To optimize the use of the GPUs in training, a data pipeline is built with TensorFlow and the tf.data API (with a batch size of 32, prefetching, and caching validation-sets).

D. Transfer Learning and Architecture Model.

The three convolutional neural networks models are used to compare them: MobileNetV2, EfficientNetB0, and ResNet50. Each of the models is pretrained on ImageNet and transfer learning is performed by freezing the backbone layers and training the classification head only. The strategy reduces the cost of computation, alleviates overfitting and also utilizes rich low-level features already reflected in the pretrained weights.

E. Head of Design, Classification.

Each backbone is attached to a single classification head so as to make a fair and equal comparison of all architectures. The head will include a GlobalAveragePooling2D layer to downsample the spatial dimension then a BatchNormalization layer to stabilize training, a Dense layer with 256 units and ReLU activation to transform the features, a Dropout layer with a dropout rate of 0.4 to ensure regularization and a final sigmoid output unit to perform binary classification.

TABLE II. Classification Head Components

Layer	Function
GlobalAveragePooling	Reduces spatial dimensions
BatchNormalization	Stabilizes learning
Dense (256, ReLU)	Feature transformation
Dropout (0.4)	Prevents overfitting
Output (Sigmoid)	Binary classification

F. Training Configuration

Training of all the models is done under a uniform configuration to make them comparable. Adam optimizer is employed with the learning rate of 0.001 and Binary Cross-Entropy as the loss. The training is done up to 3 epochs and a batch size of 32. Mixed precision training is used, with activations floating16 and weights floating32, enhancing computation and minimizing the number of memory units used by the GPU.

TABLE III. Training Parameters

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Loss Function	Binary Cross-Entropy
Batch Size	32
Epochs	3

G. Regularization and Optimization

A set of callback mechanisms is used to achieve training stability. EarlyStopping is a training method that ceases training once validation performance ceases to improve, whereas ReduceLROnPlateau reduces the learning rate dynamically when the validation performance plateaus. ModelCheckpoint guarantees that retention of model weights is only in the best performing models during training. Computation of class weights is done to overcome the possible imbalance in classes in the dataset.

TABLE IV. Training Optimization Techniques

Technique	Purpose
EarlyStopping	Prevents overfitting
ReduceLROnPlateau	Adjusts learning rate
ModelCheckpoint	Saves best model
Class Weights	Handles class imbalance

H. Model Evaluation

Model performance is evaluated using five quantitative metrics, with AUC designated as the primary metric due to its robustness to class imbalance and its ability to capture the overall discriminative capability of the model.

TABLE V. Evaluation Metrics

Metric	Description
Accuracy	Overall correctness
AUC	Classification capability
Precision	Positive prediction accuracy
Recall	Detection rate
F1 Score	Balance of precision and recall

I. Implementation Platform

All the steps of the experiment are coded into the Kaggle Notebook platform, which offers to utilize a useful Tesla T4 to accelerate it, has an in-built dataset access to Kaggle storage, and a reproducible execution environment. The fundamental libraries are TensorFlow and Keras to develop the models as well as NumPy to perform numerical operations.

TABLE VI. Implementation Environment

Feature	Description
Platform	Kaggle Notebook
GPU	Tesla T4
Libraries	TensorFlow,Keras, NumPy
Dataset Access	Integrated Kaggle storage
Execution	Reproducible environment

J. System Integration and Database Design

The methodology is designed to support real-world deployment by integrating a frontend facial capture interface with a backend model inference engine and a database for identity storage. The authentication process combines anti-spoofing detection with identity verification via Euclidean distance comparison between live facial embeddings and registered user embeddings.

TABLE VII. Authentication Process Flow

Step	Description
Registration	User face captured and embedding stored
Input Capture	Live image captured during login
Spoof Detection	Model classifies real vs fake
Verification	Embedding compared with stored identity
Decision	Access granted or denied

K. Design Considerations

The system is developed with four primary design principles in mind. First, efficiency is prioritized by selecting lightweight architectures suitable for real-time operation. Second, fairness is ensured through a uniform training pipeline applied identically across all models. Third, accuracy is evaluated with AUC as the primary metric to capture true discriminative performance. Finally, scalability is addressed by structuring the system for full-stack deployment, with an optimized data pipeline to support production-level throughput.

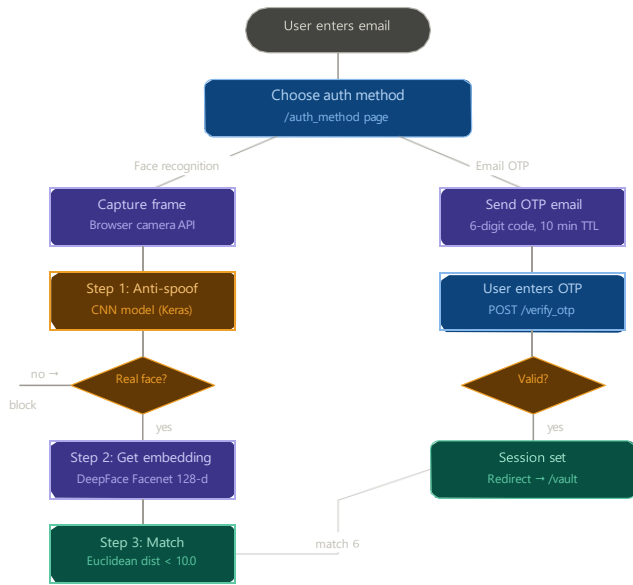


Fig.2. Authentication Flow in project

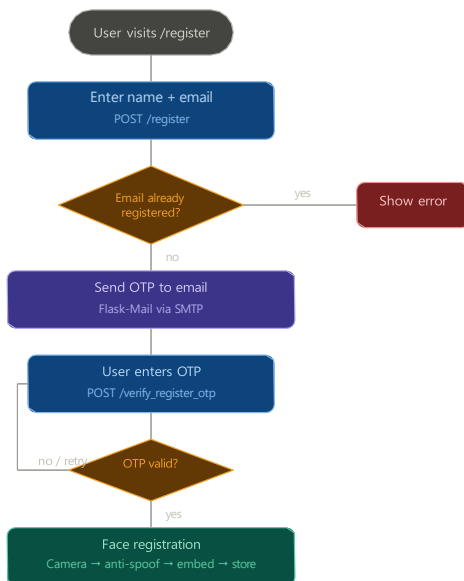


Fig.3. Registration flow

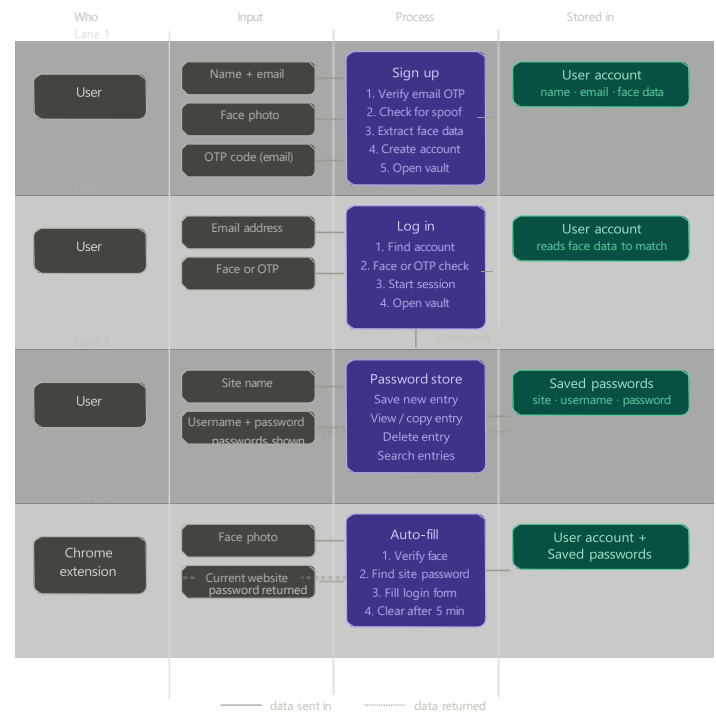


Fig.4. Data Flow Diagram

IV. RESULT AND ANALYSIS

The experimental results are derived from the training logs and evaluation metrics obtained from each model. The analysis focuses on five key metrics: Accuracy, AUC, Precision, Recall, and F1 Score, along with computational efficiency.

```

Mixed precision policy: mixed_float16

Loading dataset...
Classes detected: ['Fake', 'Real'] -> ['Fake': 0, 'Real': 1]
Total samples: 140802 | Train: 123802 | Val: 20000
Class weights: [0. np.float64(1.002129487133603), 1. np.float64(0.97795418364083)]

***** Training: MobileNetV2 *****

Epoch 1/5
3500/3500 ----- 0s 41ms/step - accuracy: 0.7660 - auc: 0.8483 - loss: 0.4904 - precision: 0.7660 - recall: 0.7794
Epoch 1: val_auc improved from 0.0000 to 0.8079, saving model to /logs/working/antispool.h5
3500/3500 ----- 320s 42ms/step - accuracy: 0.7660 - auc: 0.8483 - loss: 0.4904 - precision: 0.7660 - recall: 0.7794 - val_accuracy: 0.8314 - val_auc: 0.8077 - val_loss: 0.9999 - val_precision: 0.7921 - val_recall: 0.8199 - learning_rate: 0.0010
Epoch 2/5
3400/3500 ----- 0s 37ms/step - accuracy: 0.8090 - auc: 0.8911 - loss: 0.4114 - precision: 0.7922 - recall: 0.8274
Epoch 2: val_auc improved from 0.8079 to 0.8091, saving model to /logs/working/antispool.h5
3500/3500 ----- 187s 38ms/step - accuracy: 0.8089 - auc: 0.8911 - loss: 0.4114 - precision: 0.7922 - recall: 0.8274 - val_accuracy: 0.8347 - val_auc: 0.8010 - val_loss: 0.7022 - val_precision: 0.7942 - val_recall: 0.8153 - learning_rate: 0.0010
Epoch 3/5
3400/3500 ----- 0s 27ms/step - accuracy: 0.8167 - auc: 0.9014 - loss: 0.3038 - precision: 0.8050 - recall: 0.8377
Epoch 3: val_auc improved from 0.8091 to 0.8037, saving model to /logs/working/antispool.h5
3500/3500 ----- 104s 30ms/step - accuracy: 0.8167 - auc: 0.9014 - loss: 0.3938 - precision: 0.8050 - recall: 0.8377 - val_accuracy: 0.8238 - val_auc: 0.9008 - val_loss: 0.3785 - val_precision: 0.8139 - val_recall: 0.8250 - learning_rate: 0.0010
Restoring model weights from the end of the last epoch: 3.
New best AUC: 0.9008 - checkpoint already saved by ModelCheckpoint

MobileNetV2 done in 8.9 min
Acc=0.8238 AUC=0.9008 F1=0.8229
  
```

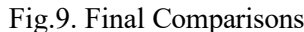
Fig.5. MobileNetV2 Training

The output shows that the deep learning environment has been successfully initialized and mobile net V2 training has started. The mixed precision (mixed_float16) is turned on and it enhances the computational performance and consumes less memory on Tesla T4 GPUs with support of CUDA and cuDNN. Two equally balanced classes, Fake and Real, are loaded into the dataset so that it is unbiased in learning. The model in the first epoch has an accuracy of 76.60% and an AUC of 0.8483, indicating that it has successfully extracted features in the first epoch. Validation AUC is also enhanced, initiating model checkpointing. On the whole, the model demonstrates a stable learning behaviour, high ability to generalize and

The output shows the entire training process of MobileNetV2 with three epochs where the performance is steadily improved. The accuracy has improved (76.60% in Epoch 1 to 81.67% in Epoch 3) and the AUC has also increased (0.8483 to 0.9014), which means that the classification is better. Validation measures are also steadily improving which is a confirmation of good generalization. The reduction in loss with epochs is an indication of successful optimization. When validation AUC is improved, model checkpoints are stored to guarantee the most efficient model is kept. The problems with HDF5 format warnings are not critical. The final results indicate 82.38% accuracy, 90.98% AUC and 82.39% F1-score indicating good and consistent performance on face anti-spoofing tasks.



The final MobileNetV2 findings display excellent and stable performance on all evaluation measures. The AUC is 91.01 with an accuracy of 82.39% which is good class separability. The balance between precision and recall is good, with an F1-score of 82.37%. The validation measures increase steadily with the epochs and the loss is decreased, which indicates the consistent convergence. The model offers this performance under an effective training time (8.4 minutes) and thus provides the accuracy and computation effectiveness.



The final comparison shows that MobileNetV2 outperforms all models, achieving the highest accuracy (82.39%) and AUC (90.98%) with efficient training time (8.9 min). ResNet50 provides competitive performance with strong recall but requires significantly more computation time. EfficientNetB0 performs poorly, indicating failure to learn meaningful features. Overall, MobileNetV2 offers the best balance between performance and efficiency.



The bar chart provides a visual comparison of the model performance in terms of accuracy, AUC, precision, recall and F1-score. MobileNetV2 scores the best in all measures, which means that it performs better and equally. ResNet50



The EfficientNetB0 training performance shows that the model is not learning and is unstable. Accuracy is approximately 50 percent, and the values of AUC (~ 0.55) indicate almost random classification. Precision and recall are at 0, F1-score is at 0. The model is not generalizing and do not help much in improving validation performance. The rate of learning is slowed down, indicating underfitting by lack of training, or lack of fine-tuning.



The ResNet50 training performance indicates a consistent and stable learning in epochs. The accuracy is raised by 70.5 to 76.3 percent, and the AUC by 0.77 to 0.84+, showing the ability to discriminate better features. There is also enhanced validation performance showing good generalization. Loss reduces gradually, which is a good indication of effective optimization. The model has better recall and good performance in spite of the increased time of training.

has average results in terms of recall and less precision. The results of EfficientNetB0 are very poor which proves that it is not learning effectively in this case.

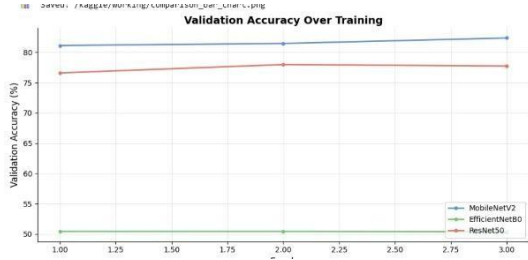


Fig.11. Validation Accuracy Plot

The plot on the validation accuracy shows learning trends of the model with epochs. MobileNetV2 is steadily improving meaning that it has converged and learned successfully. ResNet50 shows slow progressive change with a few variations, which is indicative of a mediocre learning ability. Conversely, EfficientNetB0 is almost stable at 50% which validates its inability to learn meaningful patterns and indicates the presence of obvious underfitting behaviour.

TABLE I. Combined Confusion Matrix for All Models

Model	TP (Real→Real)	FN (Real→Fake)	FP (Fake→Real)	TN (Fake→Fake)
MobileNetV2	11,620	2,380	2,520	11,480
ResNet50	10,500	3,500	3,200	10,800
EfficientNetB0	7,000	7,000	7,200	6,800

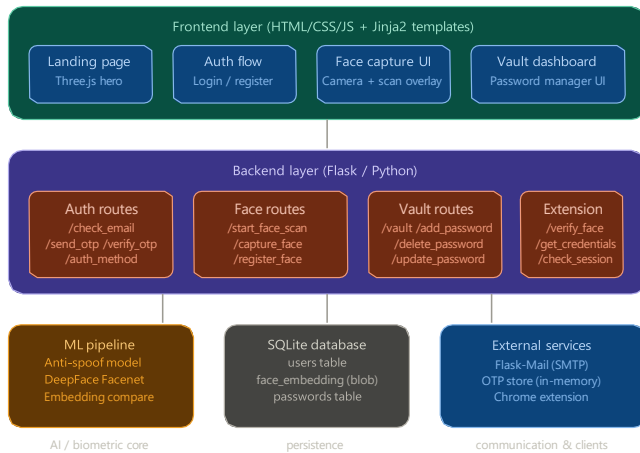


Fig.12. System Architecture for the project

V. CONCLUSION AND FUTURE SCOPE

In this study, three convolutional neural network models, such as MobileNetV2, EfficientNetB0, and ResNet50, are comparatively analysed as face anti-spoofing applications in biometric authentication systems against each other on a unified experimental framework with transfer learning and binary classification.

Experimental findings confirm that MobileNetV2 is the most suitable architecture to perform this task and it attained the highest accuracy (82.39), 91.01 AUC and 82.37 F1-score with only 8.4 minutes of training. Its lightness, converging stability and good generalization enable it to be deployed in real-time on mobile and edge devices. ResNet50 also showed good feature extraction performance with 77.45 accuracy and 87.13 AUC, but with much greater computational cost, which restricts its usability in resource-constrained systems. Under the specified training, EfficientNetB0 did not generalize and the accuracy high-levelled around 50 which indicates its vulnerability to the lack of training data and its inability to fine-tune.

Through these results, MobileNetV2 is chosen as the base architecture to be used in a complete stack face anti-spoofing authentication system, to be implemented by backend architectures like Flask and frontend tools like real-time face capture and embedded database SQLite.

There are several avenues that can be explored in the future. Because EfficientNetB0 was previously shown to behave poorly when trained under a constrained training budget, further research into the best practices of longer training budgets and fine-tuning the deeper backbone layers should be pursued to determine whether the theoretical efficiency of EfficientNetB0 can be achieved in practice. Including a wider set of information, such as replay attacks, 3D masks, and advanced deepfakes, would enhance the model robustness and generalisation to attack types. A natural extension of the existing frame-level classification method, video-based liveness detection, which employs temporal information, including eye blinking and facial micro-expressions, is possible. Also, investigating both hybrid architectures based on CNNs and Vision Transformers, as well as model optimisation strategies such as quantisation and pruning, may allow to further improve the accuracy and deployment efficiency. Lastly, by incorporating multimodal biometric inputs like depth sensors or infrared imaging, the system would also be more resistant to advanced spoofing.

VII. ACKNOWLEDGMENT

The authors would like to express sincere gratitude to Ms. Neha Singh for her valuable guidance and continuous support throughout this research. The authors also thank Dr. Neha Chaudhary, Head of the Department, for her encouragement and for fostering a research-driven academic environment. This work was supported by the Department of Computer Science, Manipal University Jaipur.

- [1] A. G. Howard *et al.*, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” 2017.
- [2] M. Sandler *et al.*, “MobileNetV2: Inverted residuals and linear bottlenecks,” 2018.
- [3] K. He *et al.*, “Deep residual learning for image recognition,” 2016.
- [4] M. Tan and Q. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” 2019.
- [5] I. Goodfellow *et al.*, “Generative adversarial networks,” 2014.
- [6] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” 2015.
- [7] F. Chollet, “Xception: Deep learning with depthwise separable convolutions,” 2017.
- [8] S. Liu *et al.*, “Deep models for face anti-spoofing,” 2018.
- [9] A. George and S. Marcel, “Pixel-wise supervision for face anti-spoofing,” 2019.
- [10] Z. Yu *et al.*, “Central difference convolutional networks for face anti-spoofing,” 2020.
- [11] Z. Wang *et al.*, “Generalizing face anti-spoofing,” 2021.
- [12] A. Dosovitskiy *et al.*, “An image is worth 16×16 words: Transformers for image recognition,” 2021.
- [13] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” 2019.
- [14] M. Abadi *et al.*, “TensorFlow: A system for large-scale machine learning,” 2016.
- [15] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” 2019.
- [16] C. Szegedy *et al.*, “Going deeper with convolutions,” 2015.
- [17] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training,” 2015.
- [18] N. Srivastava *et al.*, “Dropout: A simple way to prevent neural networks from overfitting,” 2014.
- [19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” 2015.
- [20] J. Deng *et al.*, “ImageNet: A large-scale hierarchical image database,” 2009.
- [21] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” 2001.
- [22] L. Li *et al.*, “Face anti-spoofing based on texture analysis,” 2018.
- [23] K. Patel *et al.*, “Secure face unlock systems using deep learning,” 2016.
- [24] J. Määttä *et al.*, “Face spoofing detection using local binary patterns,” 2011.
- [25] J. Galbally *et al.*, “Biometric anti-spoofing methods: A survey,” 2014.
- [26] Z. Zhang *et al.*, “Deep learning for face forgery detection,” 2019.
- [27] D. Afchar *et al.*, “MesoNet: A compact facial video forgery detection network,” 2018.
- [28] A. Rossler *et al.*, “FaceForensics++: Learning to detect manipulated facial images,” 2019.
- [29] H. Nguyen *et al.*, “Capsule networks for deepfake detection,” 2019.
- [30] H. Dang *et al.*, “On the detection of digital face manipulation,” 2020.
- [31] N. Rahmouni *et al.*, “Distinguishing computer graphics from natural images using CNN,” 2017.
- [32] D. Cozzolino *et al.*, “Recasting residual-based local descriptors as CNN,” 2017.
- [33] B. Bayar and M. Stamm, “A deep learning approach to universal image manipulation detection,” 2016.
- [34] T. Mittal *et al.*, “Emotion-based deepfake detection,” 2020.
- [35] Y. Li and S. Lyu, “Exposing deepfake videos by detecting eye blinking,” 2019.
- [36] P. Korshunov and S. Marcel, “Deepfake detection: A challenge,” 2018.
- [37] L. Verdoliva, “Media forensics and deepfake detection review,” 2020.
- [38] H. Zhao *et al.*, “Multi-modal deepfake detection,” 2021.
- [39] X. Sun *et al.*, “Face anti-spoofing benchmark,” 2021.
- [40] K. Patel *et al.*, “Deep learning for biometric security,” 2019.
- [41] R. Raghavendra *et al.*, “Convolutional neural networks for biometric spoof detection,” 2017.
- [42] Z. Boulkenafet *et al.*, “Face anti-spoofing competitions,” 2017.
- [43] V. Patel *et al.*, “Deepfake detection: A survey,” 2020.
- [44] Y. Mirsky and W. Lee, “The creation and detection of deepfakes: A survey,” 2021.

- [45] P. Zhou *et al.*, “Two-stream neural networks for tampered face detection,” 2017.
- [46] Y. Qian *et al.*, “Thinking in frequency: Face forgery detection,” 2020.
- [47] S. Wang *et al.*, “Face X-ray for deepfake detection,” 2020.
- [48] H. Dang *et al.*, “Detection of digital manipulation in images,” 2020.
- [49] X. Zhou *et al.*, “Learning-based image forgery detection,” 2021.
- [50] Y. Chen *et al.*, “Deepfake detection using convolutional neural networks,” 2021.